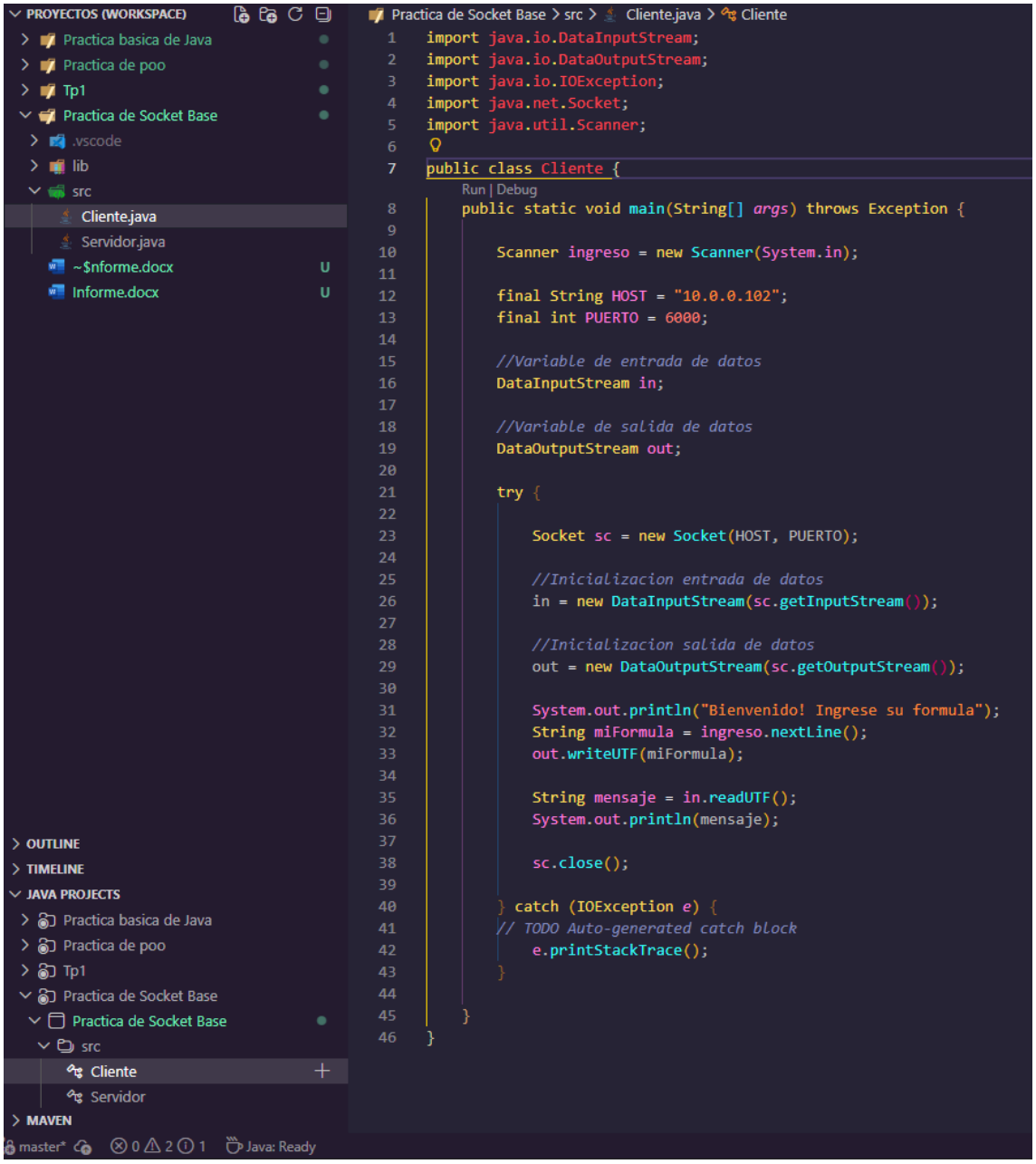
Los sockets se utilizan como el puente entre el cliente y el servidor, transportan la información que uno envía al otro



PROYECTOS (WORKSPACE)

- Practica basica de Java
- Practica de poo
- Tp1
- Practica de Socket Base
  - .vscode
  - lib
  - src
    - Cliente.java
    - Servidor.java
  - ~$nforme.docx U
  - Informe.docx U

Practica de Socket Base > src > Servidor.java > ...

```java
import java.io.DataInputStream;
import java.io.DataOutputStream;

import java.io.IOException;
import java.net.ServerSocket;
import java.net.Socket;

import java.util.ArrayList;
import java.util.List;


public class Servidor {

    Run | Debug
    public static void main(String[] args) throws Exception{

        ServerSocket servidor = null;
        Socket miSocket = null;

        //Variable de entrada de datos
        DataInputStream in;

        //Variable de salida de datos
        DataOutputStream out;

        final int PUERTO = 6000;


            try {

                servidor = new ServerSocket(PUERTO);
                System.out.println("Servidor inciado, esperando al cliente");

                boolean conectado = true;
                while (conectado) {
                    miSocket = servidor.accept();
                    System.out.println("Cliente conectado");

                    System.out.println("Esperando instrucciones del cliente");

                    //Inicializacion entrada de datos
                    in = new DataInputStream(miSocket.getInputStream());
                    //Inicializacion salida de datos
                    out = new DataOutputStream(miSocket.getOutputStream());

                    //Lee el mensaje enviado por el cliente
                    String mensaje = in.readUTF();

                    if (mensaje.trim().equalsIgnoreCase("SALIR")) {
                        System.out.println("Petición de desconexión del cliente");
                        servidor.close();
                        conectado = false;
                    } else if (mensaje.trim().toUpperCase().startsWith("RESOLVE")){
                        String resultado = calculadoraFunciones(mensaje);
                        out.writeUTF("Resultado de la operacion: " + resultado);
                    } else {
                        out.writeUTF("Mensaje recibido: " +mensaje);
                    }

                    miSocket.close();

                    System.out.println("Cliente desconectado");
                }

            } catch (IOException ex) {
                // TODO Auto-generated catch block
                ex.printStackTrace();
            }
    }
```

OUTLINE

TIMELINE

JAVA PROJECTS

- Practica basica de Java
- Practica de poo
- Tp1
- Practica de Socket Base
  - Practica de Socket Base
    - src
      - Cliente
      - Servidor
    - JRE System Library [JavaSE-25]
    - Referenced Libraries

```java
    // Extrae la ecuación de las comillas y llama al evaluador
    private static String calculadoraFunciones(String input) {
        try {
            int primerComilla = input.indexOf('"');
            int ultimaComilla = input.lastIndexOf('"');

            if (primerComilla == -1 || ultimaComilla == -1 || primerComilla == ultimaComilla) {
                return "Error de sintaxis. Usa: RESOLVE \"45*23/54+234\"";
            }

            // Extrae solo lo que está entre comillas y quita espacios en blanco
            String ecuacion = input.substring(primerComilla + 1, ultimaComilla).replace(" ", "");
            double resultado = resolverEcuacion(ecuacion);

            return String.valueOf(resultado);

        } catch (Exception e) {
            return "Error al calcular. Verifica que solo haya números y operadores básicos.";
        }
    }

    // Evaluador intermedio: prioriza multiplicaciones/divisiones
    private static double resolverEcuacion(String ecuacion) {
        List<Double> numeros = new ArrayList<>();
        List<Character> operadores = new ArrayList<>();

        // 1. Parsear el string para separar números de operadores
        StringBuilder tempNum = new StringBuilder();
        //Recorre toda la ecuacion
        for (int i = 0; i < ecuacion.length(); i++) {
            char c = ecuacion.charAt(i);
            if (Character.isDigit(c) || c == '.') {
                tempNum.append(c);
            } else if (c == '+' || c == '-' || c == '*' || c == '/') {
                numeros.add(Double.parseDouble(tempNum.toString()));
                operadores.add(c);
                tempNum.setLength(0); // Limpiar el buffer numérico
            }
        }
        numeros.add(Double.parseDouble(tempNum.toString())); // Agregar el último número

        // 2. Primera pasada: Resolver multiplicaciones y divisiones
        for (int i = 0; i < operadores.size(); i++) {
            char op = operadores.get(i);
            if (op == '*' || op == '/') {
                double a = numeros.get(i);
                double b = numeros.get(i + 1);
                double resultado = (op == '*') ? (a * b) : (a / b);

                numeros.set(i, resultado); // Reemplazar el primer número por el resultado
                numeros.remove(i + 1);     // Borrar el segundo número
                operadores.remove(i);      // Borrar el operador usado
                i--; // Retroceder el indice porque las listas se achicaron
            }
        }

        // 3. Segunda pasada: Resolver sumas y restas de izquierda a derecha
        double resultadoFinal = numeros.get(0);
        for (int i = 0; i < operadores.size(); i++) {
            char op = operadores.get(i);
            double sigNumero = numeros.get(i + 1);
            if (op == '+') resultadoFinal += sigNumero;
            if (op == '-') resultadoFinal -= sigNumero;
        }

        return resultadoFinal;
    }
}
```

*Agustin Estrella – Legajo: 114539*